Project - High Level Design

On

Dockerized Marketing PHP Laravel Services

Course Name: Devops Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	CHARU RATHOD	EN23CS3T1007
2.	RUDRAKSH SHARMA	EN22CS301832
3.	PRATHAM DESHMUKH	EN22CS301741
4.	PIYUSH PATIDAR	EN22CS301705
5.	ROHAN JAIN	EN22CS301817
6.	PRIYANSH MEWADA	EN22CS301763

Group Name: 03D7

Project Number: DO-26

Industry Mentor Name: Mrs.Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table Content

1. INTRODUCTION

The main objective of this project is to develop a **marketing automation system** using PHP Laravel and deploy it using Docker containers.

In traditional development environments, developers often face issues such as:

- Inconsistent environments across systems
- Dependency conflicts
- Time-consuming setup processes

To overcome these challenges, Docker is used. Docker ensures:

- Consistent environments across development, testing, and production
- Faster deployment
- Easy scalability

Laravel is chosen as the backend framework because it provides:

- A structured MVC architecture
- Built-in security features
- Rapid development capabilities

1.1. Scope of the document

This document provides a high-level overview of the system design and architecture. It includes:

- System components and architecture
- Data flow and process flow
- API structure
- Security and performance considerations

This document helps in understanding how the system works and how different components interact.

1.2. Intended Audience

This document is designed for:

- Developers (Backend and DevOps Engineers)
- System Architects
- Test Engineers
- Project Evaluators and Faculty

It helps each group understand the system from their perspective.

1.3. System overview

The system is a **marketing platform** that allows users to:

- Create and manage marketing campaigns
- Select target audiences
- Send notifications via email or SMS
- Track campaign performance using analytics

Internally:

- Laravel handles application logic
- Docker manages deployment
- Redis handles background job processing

2.System Design

The system is designed using a modular architecture:

- Presentation Layer → Admin dashboard (UI)
- Application Layer → Laravel backend logic
- Data Layer → MySQL database
- Infrastructure Layer → Docker containers

Each component runs in a separate container:

- Laravel Application Container
- Nginx Web Server Container
- MySQL Database Container
- Redis Container

This design ensures flexibility, scalability, and isolation.

2.1. Application Design

The application follows the **MVC (Model-View-Controller)** pattern:

- **Model** → Handles database operations.
- **View** → Displays data to the user.
- **Controller** → Processes user requests and business logic.

Example:

When a user creates a campaign:

- The Controller processes the request.
- The Model stores data in the database.
- The View shows confirmation to the user.

2.2. Process Flow

The system workflow is as follows:

1. User logs into the system
2. User creates a marketing campaign
3. User selects the target audience
4. The system stores campaign details
5. Notification tasks are added to a queue
6. Background workers process the queue
7. Notifications are sent
8. Analytics data is collected and displayed

This ensures efficient handling of large workloads.

2.3. Information Flow

The data flows through the system in the following manner:

- User Input → Controller → Service Layer → Database
- Campaign Data → Queue System → Worker → Notification Service
- Analytics Data → Stored → Displayed on Dashboard

This structured flow ensures data consistency and reliability.

2.4. Components Design

- The system consists of the following components:
- Laravel Application
- Nginx Web Server
- MySQL Database
- Redis Cache and Queue
- Background Worker

All components communicate through a Docker network, ensuring smooth interaction.

2.5. Key Design Considerations

While designing the system, the following factors were considered:

- Scalability → Ability to handle increasing users
- Performance → Fast response times
- Security → Protection of user data
- Maintainability → Easy updates and debugging

2.6. API Catalogue

The system exposes RESTful APIs for communication.
Examples:

- POST /login → User authentication
- POST /campaign → Create a campaign
- GET /campaigns → Retrieve campaigns
- POST /send → Send notifications

All APIs follow standard HTTP methods and return JSON responses.

3. Data Design

Data Design defines how data is represented and handled within the system.
The system uses lightweight in-memory data structures to store goals, tasks, and schedules during execution.

3.1 Data Model

The relationships between entities are:

- One User → Multiple Campaigns
- One Campaign → Multiple Notifications

This relational model ensures efficient data organization.

3.2 Data Access Mechanism

Laravel's **Eloquent ORM** is used for database interaction.

Benefits:

- Simplifies database queries
- Improves security
- Reduces development time

3.3 Data Retention Policies

The system defines how long data is stored:

- Important data → Stored permanently
- Logs → Stored temporarily
- Old analytics → Archived periodically

3.4 Data Migration

Database schema changes are managed using Laravel migrations.

Features:

- Version control of database
- Easy updates
- Rollback capability

4. Interfaces

The system provides multiple interfaces:

- Web-based Admin Dashboard
- REST APIs for integration
- Third-party services (Email/SMS APIs)

5. State and Session Management

User sessions are managed using:

- Laravel session management
- Cookies
- Redis (optional for scalability)

This ensures users remain authenticated during interactions.

6. Caching

Caching is implemented using Redis.

Benefits:

- Faster data retrieval
- Reduced database load
- Improved performance

7. Non-Functional Requirements

These define system quality attributes:

- Performance
- Scalability
- Reliability
- Maintainability

7.1 Security Aspects

Security measures include:

- User authentication and authorization
- Data encryption
- Protection against SQL Injection and XSS
- Secure API communication

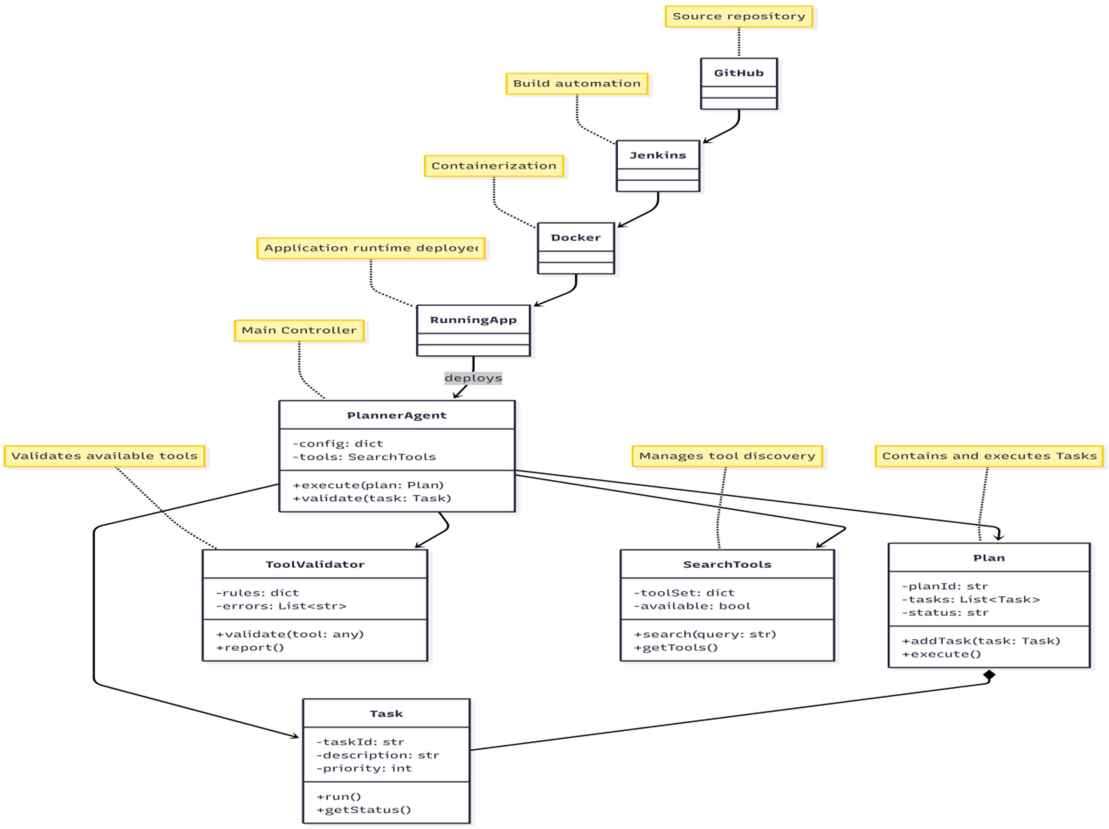
7.2 Performance Aspects

Performance is optimized using:

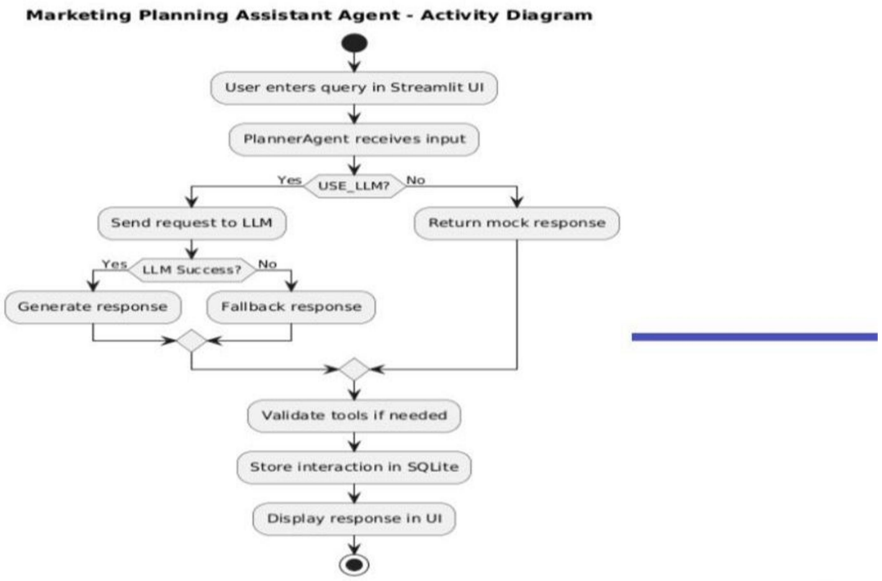
- Queue-based processing
- Caching
- Optimized database queries
- Load balancing (if scaled)

8. Diagram & Flowcharts :

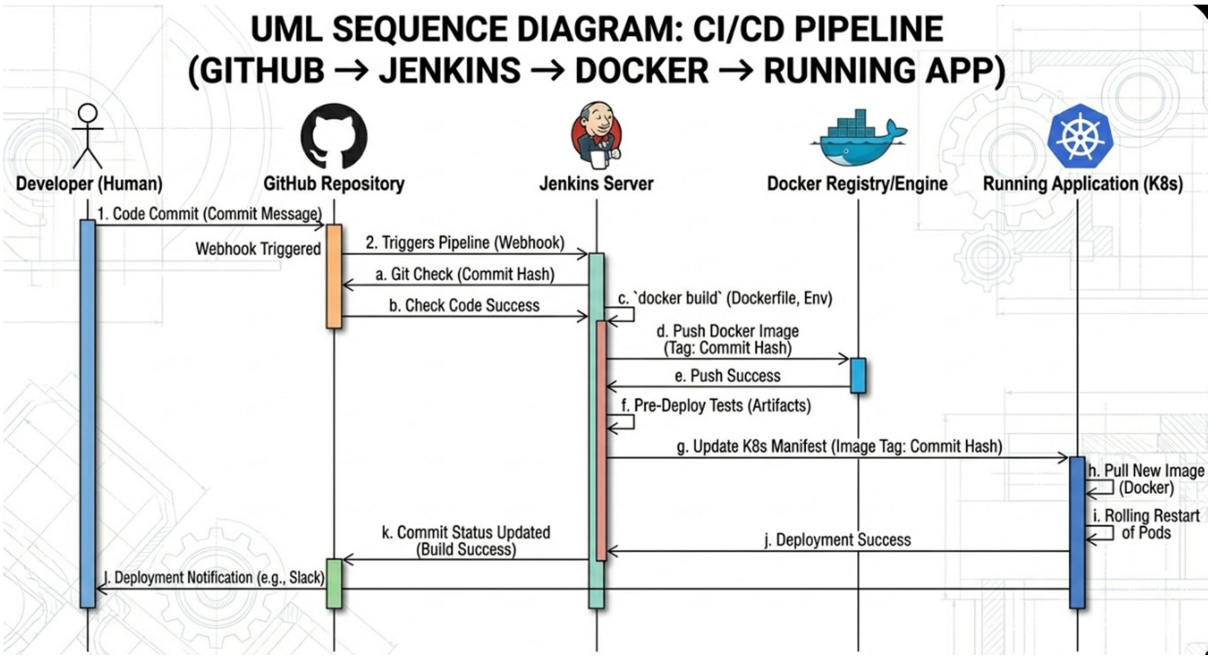
8.1 Class Diagram



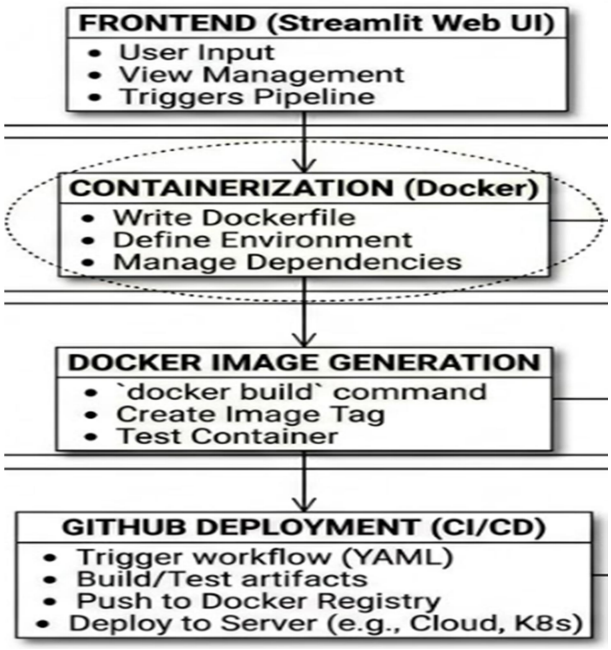
8.2 Activity Diagram



8.3 Sequence Diagram



8.4 Component Diagram



References

- The references section lists the technical resources and documentation used for understanding and designing the system.
- Laravel Official Documentation
- Docker Documentation
- REST API Design Guidelines
- DevOps Best Practices